

Integración de Soluciones para Plataformas Cloud

Semana 2 · Patrones de integración: API, mensajería y eventos

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semana 2 · Periodo 2026-2 · Barranquilla, Colombia

 Google Cloud

De la infraestructura a la integración

La semana pasada definimos **dónde** corren los sistemas (multicloud vs. híbrido). Hoy empezamos a responder **cómo se comunican entre sí**.

Un sistema cloud moderno es una **conversación entre servicios**: pedidos, notificaciones, reacciones a cambios.

Existen exactamente **tres patrones fundamentales** para esa conversación; todo lo demás (Kafka, EventBridge, service bus) son implementaciones de estos tres.

API

pregunta → respuesta

Mensajería

cola: enviar y olvidar

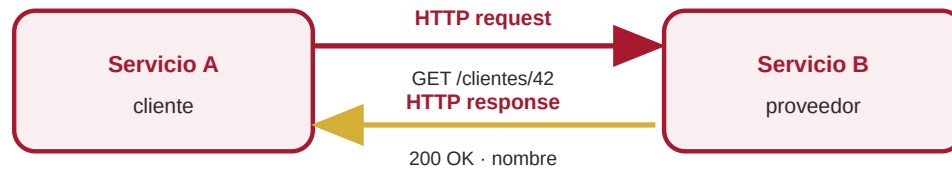
Eventos

pub/sub: avisar a todos

El mismo vocabulario existe en TODAS las nubes

AWS SQS/SNS · Azure Service Bus/Event Grid · Google Pub/Sub

Patrón 1: Integración vía API (síncrona)



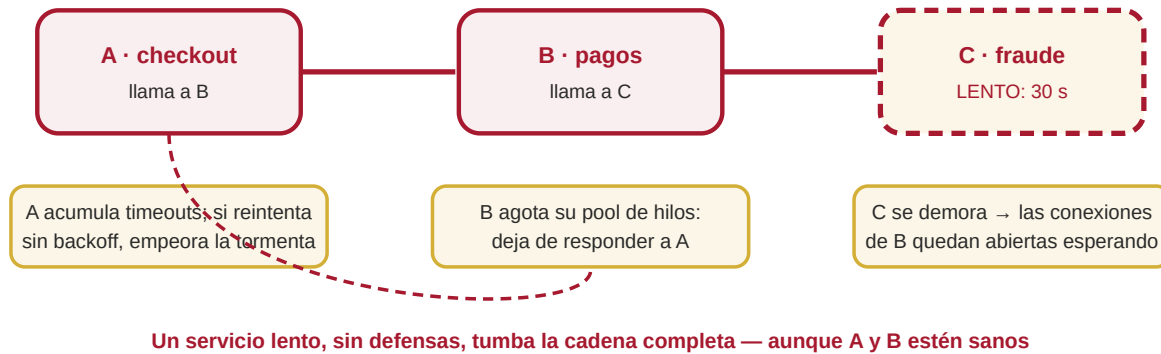
Ambos servicios deben estar **despiertos al mismo tiempo**

Cuándo usarlo: necesitas la respuesta **ahora** para continuar: consultar saldo, validar un token, confirmar inventario.

Riesgo — acoplamiento temporal: si B cae o va lento, A falla o se bloquea; en picos hay que escalar ambos a la vez.

Caso real — Netflix (2012): una dependencia lenta agotó los hilos del API y la falla se propagó en cascada; la respuesta fue **Hystrix**: circuit breaker y «bulkhead» por dependencia.

API bajo presión: anatomía de una caída en cascada

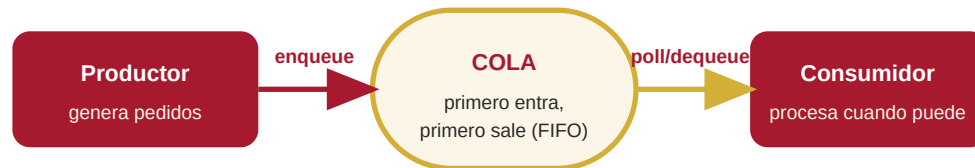


Lección de Netflix (2012): el API moría por dependencias lentas; nació **Hystrix** para aislar cada dependencia remota.

Las tres defensas: timeout corto siempre · **circuit breaker** (dejar de llamar al que está caído) · **bulkhead** (pool separado por dependencia).

Y si no necesitas la respuesta ya... no uses API: ese es exactamente el caso para la cola de la siguiente diapositiva.

Patrón 2: Integración vía mensajería (colas)



Desacoplamiento temporal

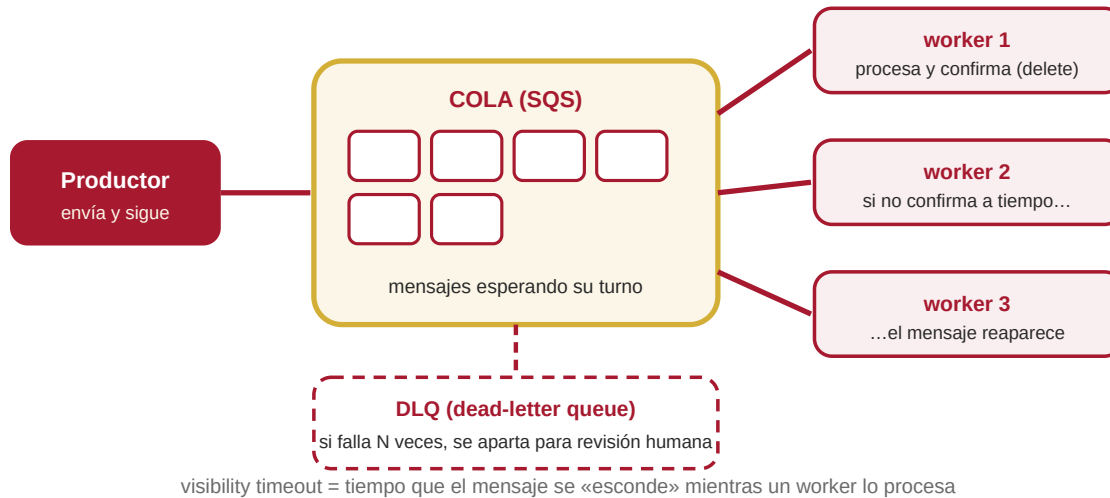
el productor ni se entera si el consumidor está caído u ocupado:
el mensaje espera en la cola hasta que alguien lo procese

Cuándo usarlo: tareas que pueden esperar unos segundos: facturar, enviar correo, redimensionar imágenes, procesar pagos por lotes.

Absorbe picos: Black Friday produce 10× mensajes; la cola los retiene y el consumidor procesa a su ritmo — nada se pierde.

Caso real — Nubank (Brasil): casi toda la comunicación entre sus microservicios es asíncrona sobre un broker replicado: el productor publica y sigue; el consumidor procesa cuando puede.

La cola por dentro: reintentos, visibilidad y DLQ

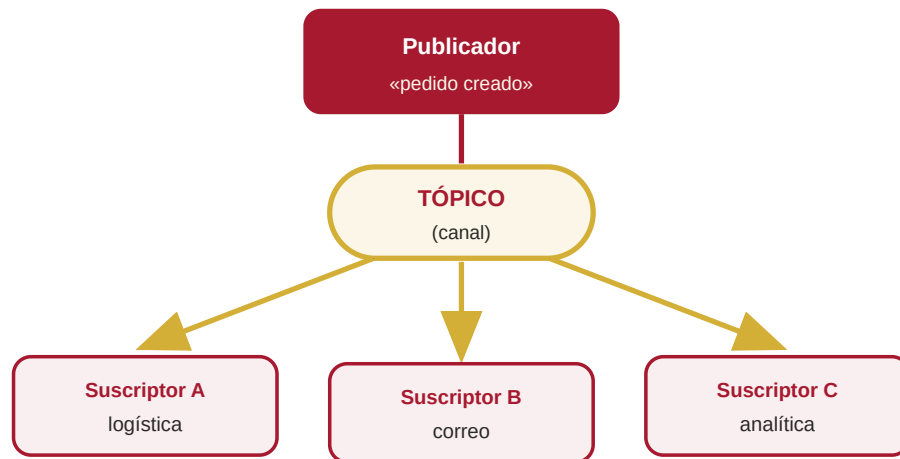


«Exactly-once» no existe: las colas reales son **at-least-once** — tu consumidor debe ser **idempotente** (procesar dos veces sin duplicar efecto).

La DLQ es tu red de seguridad: el mensaje «envenenado» no bloquea la cola ni se pierde: queda aislado con su evidencia.

En tu lab: SQS retiene mensajes hasta **14 días** y escala sola — por eso absorbió tu prueba sin configurar nada extra.

Patrón 3: Integración vía eventos (pub/sub)

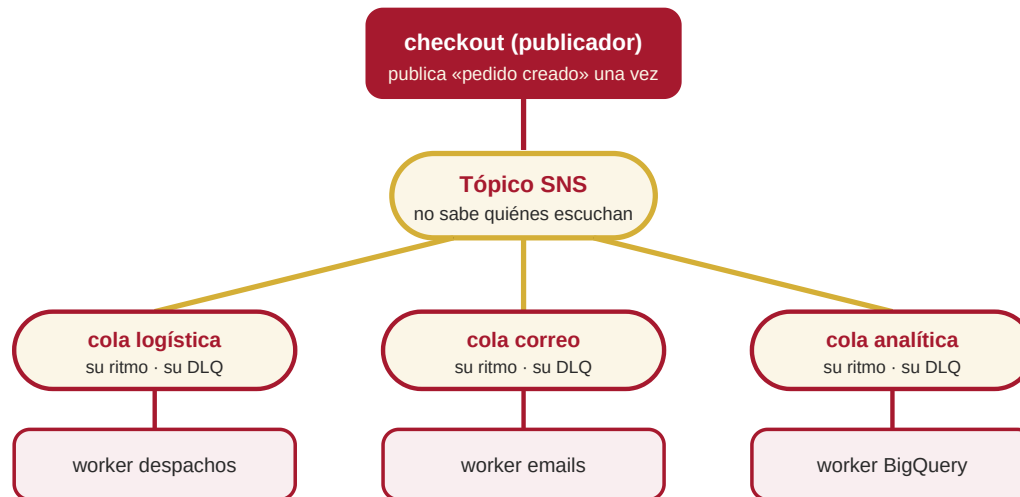


Cuándo usarlo: un hecho relevante interesa a **muchos sistemas**: cliente creado, pago recibido, inventario bajo.

Fan-out: el emisor publica una vez al tópico; cada suscriptor recibe su copia. Agregar un consumidor nuevo **no toca el código** del emisor.

Caso real — Grab: su plataforma de eventos sobre Kafka procesa **+300.000 millones de eventos por semana**; cada viaje dispara eventos a decenas de sistemas sin que el emisor los conozca.

El combo favorito de producción: fan-out SNS → SQS



Por qué una cola por suscriptor: cada consumidor procesa a **su propio ritmo**, con sus reintentos y su DLQ — si analítica cae, logística ni se entera.

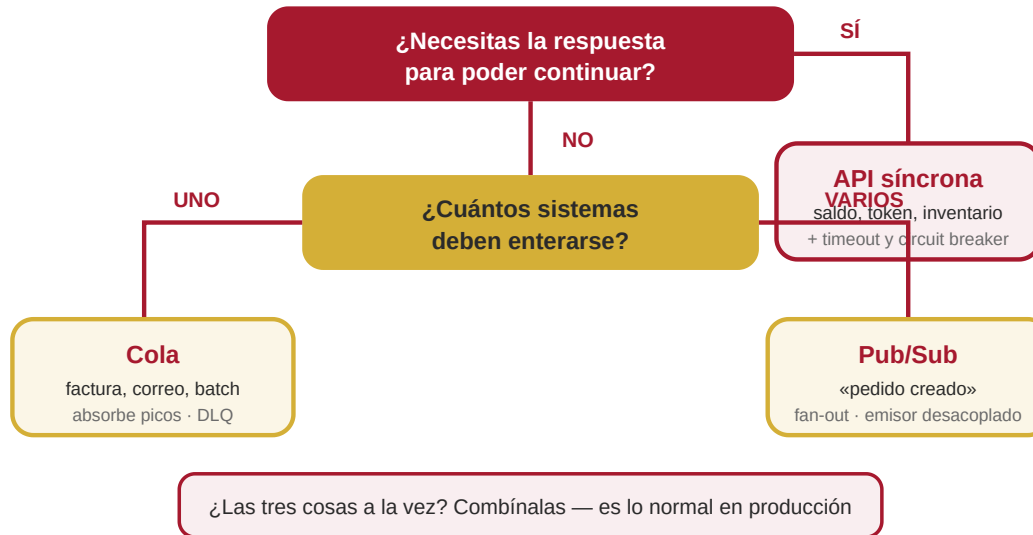
Extender sin dolor: agregar «puntos de fidelidad» mañana = suscribir una cola nueva al tópico. Cero cambios en el checkout.

Ya lo tienes montado: es exactamente lo que construiste en la Actividad 1 (SNS → SQS) del Learner Lab.

Criterio	API síncrona	Mensajería (cola)	Eventos (pub/sub)
Acoplamiento	Fuerte: ambos activos a la vez	Débil en tiempo: el mensaje espera	Nulo: el emisor no conoce receptores
¿Quién recibe?	Uno (y exactamente uno)	Uno: el primero que tome el mensaje	Todos los suscriptores del tópico (fan-out)
Entrega	Inmediata o error	Diferida pero garantizada (reintentos)	Inmediata, por copia a cada suscriptor
Si el receptor está caído	Falla al instante: timeout o error 5xx para el llamador	El mensaje espera en la cola y se procesa cuando vuelva (reintentos; DLQ si nunca vuelve)	Los suscriptores sanos reciben su copia; el caído la pierde salvo suscripción durable/DLQ
Ejemplo AWS	API Gateway + Lambda	SQS	SNS · EventBridge
Úsalo para	Consultas y validaciones en línea	Trabajo diferido y absorción de picos	Notificar a muchos sistemas de un hecho

En un sistema real se combinan: el checkout consulta inventario (**API**), encola la factura (**cola**) y dispara el evento «pedido creado» (**pub/sub**) que avisan a logística, correo y analítica a la vez.

¿Cuál patrón elegir? El árbol de decisión



La pregunta que manda es la primera: si tu flujo se detiene sin respuesta, ninguna cola te salva — es API (con defensas).

Segunda pregunta: si el trabajo puede esperar y tiene **un** dueño → cola; si el hecho le interesa a **muchos o a futuros desconocidos** → tópico.

Trampa de examen y de vida: pub/sub directo a un proceso sin cola pierde mensajes si ese proceso está caído — por eso existe el fan-out SNS → SQS.

Lo que ustedes ya construyeron: los 3 patrones en AWS

Actividad 1 (5%, U1): cada estudiante creó en su Learner Lab una **cola SQS** y un **tópico SNS** conectando dos servicios simples.

Paso del laboratorio	Patrón que ejercita	Pregunta clave de control
Crear cola SQS, enviar/recibir mensaje	Mensajería asíncrona	¿Qué pasa si borro el consumidor antes de recibir?
Crear tópico SNS + suscripción email/cola	Eventos pub/sub	¿Cuántos destinatarios reciben una sola publicación?
Llamar una API pública desde CloudShell (curl)	API síncrona	¿Qué devuelve si el servicio responde lento o cae?
Conectar SNS → SQS (fan-out a colas)	Híbrido eventos+mensajería	¿Por qué esta combinación es tan común en producción?

Idea central: dominan ahora el **vocabulario**. La semana 9 lo usaremos de verdad: los microservicios del lab AWS se coordinan con estos mismos mecanismos.

Caso real: un servicio ancla en las TRES nubes



Database@AWS

GA jul-2025

20 regiones vivas (may-2026)

Exadata dentro de datacenters AWS



Database@Azure

Primero de la familia (dic-2023)

33 regiones: la más extendida



Google Cloud

Database@Google Cloud

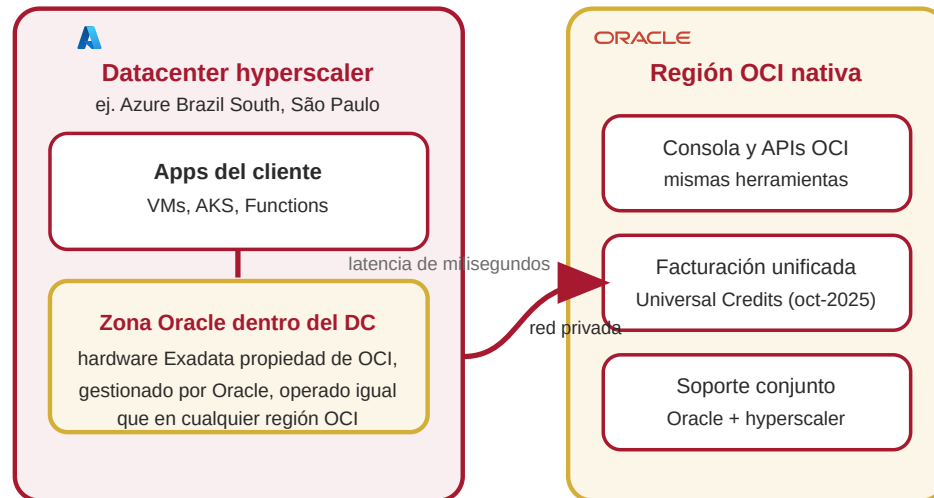
GA sep-2024

15 regiones; São Paulo viva (nov-2025)

¿Por qué importa para este curso? Es la prueba de que los proveedores ya no compiten solo contra el otro: también **se integran entre sí** — hardware Oracle gestionado por OCI corriendo físicamente dentro del datacenter del competidor, vendido en su marketplace.

Clientes que ya lo usan: Best Buy, State Street, Vodafone y Fidelity (@AWS) · MSCI, Sanofi y Conduent (@Azure) · Dun & Bradstreet (@Google Cloud).

¿Cómo funciona Database@X por dentro?



Clave técnica: NO es una réplica ni una API compatible: es el **mismo hardware Exadata**, gestionado por OCI, instalado físicamente en el datacenter del socio.

Resultado: apps del hyperscaler hablan con Oracle Database sin salir de su nube: latencia mínima y **cero costo de egress** entre ambos.

Para el negocio: modernizar la base sin re-plataformar la app, comprando todo en un solo contrato.

Vigencia verificada 2026: expansión acelerada

Familia	Hito clave	Estado 2026
Database@Azure	Disponible dic-2023, primer acuerdo del programa	Pasó de 14 a 33 regiones durante 2025 ; Exadata Exascale (mar-2025) baja el costo mínimo hasta 95%
Database@Google Cloud	GA sep-2024	15 regiones vivas (ago-2026): São Paulo desde nov-2025; México y Turín planificadas
Database@AWS	Preview dic-2024; GA jul-2025 (EE.UU.)	20 regiones (may-2026) : Europa, APAC y São Paulo incluidas
Comercial	Multicloud Universal Credits	Anunciado oct-2025 (Oracle AI World): un solo crédito consumible en OCI y las tres nubes

Fuentes verificadas ago-2026: aws.amazon.com/whats-new (may-2026), docs.oracle.com (regiones @Google Cloud), blogs.oracle.com (@Azure), oracle.com/news (Universal Credits). El patrón «servicio ancla multi-nube» pasó de experimento a catálogo completo en menos de 3 años.

Esto pasa en Latinoamérica, ahora mismo

Las tres nubes con Oracle Database dentro, hoy:



Google Cloud

Hyperscaler	Zona LatAm con Oracle dentro	Par OCI asociado	Estado
AWS	SA-East-1 (São Paulo, 2 zonas)	Brazil Southeast (Vinhedo)	Vivo desde may-2026
Azure	Brazil South (São Paulo) · Brazil Southeast (Río)	Vinhedo / Rio de Janeiro	Vivo (Brazil South desde 2024)
Google Cloud	southamerica-east1 (Osasco, São Paulo)	Brazil East (São Paulo)	Vivo desde nov-2025

¿Por qué les debería importar? Residencia de datos: bancos, gobierno y salud en Brasil/Colombia pueden mantener sus datos Oracle físicamente en Sudamérica mientras corren apps en el hyperscaler que prefieran.

Conexión con Colombia: regiones nube en el país siguen limitadas; el patrón multicloud co-ubicado amplía opciones sin esperar un datacenter local propio de cada proveedor.

La próxima clase (semana 3) explica justamente cómo se conectan físicamente esas zonas: VCN, gateways, peering, VPN y FastConnect/Interconnect.

Conclusiones clave de la semana

1. Todo sistema distribuido conversa con solo tres patrones: **API síncrona**, **mensajería con cola** y **eventos pub/sub**; el resto son variantes.
2. Elegir patrón = decidir cuánto **acoplamiento** toleras: respuesta inmediata, trabajo diferido, o difusión a desconocidos.
3. Ustedes ya ejecutaron los tres en AWS real (SQS, SNS, API); ese vocabulario vuelve en U2 con los microservicios del lab.
4. Los proveedores se están integrando entre sí: Oracle Database corre dentro de AWS/Azure/GCP con presencia **viva en São Paulo (las tres) y Río (Azure)**.
5. La integración física de esas zonas exige redes privadas entre proveedores: el tema exacto de la semana 3.

Traigan la próxima clase: repaso rápido de qué es una IP privada, una subnet y un firewall — es todo lo que necesitan para entender VCN e interconnect.